

M. S. Ramaiah Institute of Technology
(Autonomous Institute, Affiliated to VTU)
Department of Computer Science and Engineering

CSP81:Project Work

Credit:14

Project - System Design

Title: Design and Evaluation of a Noise-Aware Variational Quantum Classifier for Parkinson's Disease Detection

Project Guide Name: Dr. Geetha J

USN	Name
1MS22CS063	Hoor Parvaiz
1MS22CS069	Ishika Mohol
1MS22CS114	Rifah Balquees
1MS22CS141	Shrinidhi Pawar

Guide Signature: _____

Guide Comment: _____

1. Introduction to the System

Parkinson's disease is a progressive neurological disorder that affects movement, speech, and motor coordination. Early detection of Parkinson's disease is critical for improving treatment outcomes and slowing disease progression. However, building reliable machine learning models for early diagnosis is challenging because biomedical datasets are often limited in size due to privacy constraints, high data collection costs, and limited confirmed patient cases. Traditional machine learning models may struggle with such small datasets, often leading to issues such as overfitting and unstable predictions.

Recent advances in **Quantum Machine Learning (QML)** have introduced new approaches for handling complex data representations using quantum computing principles. Variational Quantum Circuits (VQCs), which are parameterized quantum circuits optimized using classical algorithms, have emerged as a promising model for classification tasks. These circuits can represent high-dimensional feature spaces and may potentially offer advantages in scenarios where classical models struggle with limited data. However, current quantum computers operate in the **Noisy Intermediate-Scale Quantum (NISQ)** era, where quantum computations are affected by hardware noise, decoherence, and gate errors. As a result, evaluating quantum machine learning models under realistic noisy conditions is essential for understanding their practical feasibility.

This project proposes a **Hybrid Classical-Quantum Machine Learning System for Parkinson's Disease Detection** that integrates classical data processing techniques with quantum circuit-based classification. In the proposed system, the Parkinson's dataset undergoes classical preprocessing steps including feature scaling and dimensionality reduction using Principal Component Analysis (PCA). These steps reduce the high-dimensional biomedical features into a smaller feature space compatible with quantum circuits. The processed features are then encoded into quantum states and passed through a **Variational Quantum Classifier**, where trainable quantum circuits learn to distinguish between Parkinson's and healthy samples. The hybrid model combines classical optimization methods with quantum circuit operations, forming a quantum-classical learning loop.

A key focus of this project is the **robustness analysis of the hybrid quantum model under simulated quantum noise conditions**. Since most current quantum experiments are performed using simulators rather than real quantum hardware, the system incorporates artificially generated noise models that mimic real quantum device behavior. Various types of quantum noise, such as depolarizing noise, bit-flip errors, phase-flip errors, and amplitude damping, are introduced during circuit execution to study their impact on model performance. By evaluating the model under both ideal (noise-free) and noisy simulation environments, the system provides insights into how quantum noise affects the reliability of quantum machine learning models.

To ensure a comprehensive evaluation, the system also trains classical baseline models such as Support Vector Machines and Random Forest classifiers. The performance of classical and hybrid models is analyzed using standard classification metrics including accuracy, precision, recall, F1-score, and ROC-AUC. Additionally, statistical measures such as mean accuracy, variance, and stability across multiple experimental runs are computed to assess the reliability of the models when working with small datasets.

The system is implemented as a **Python-based research prototype** that integrates classical machine learning libraries with quantum computing frameworks such as Qiskit and PennyLane. It provides a structured experimental pipeline for dataset preprocessing, hybrid model training, quantum noise simulation, performance evaluation, and visualization of results. The primary users of the system are researchers and students interested in exploring the feasibility of quantum machine learning techniques in healthcare analytics.

Overall, this project aims to provide a systematic experimental framework for studying **hybrid classical–quantum machine learning models under realistic noisy quantum conditions**. By analyzing the robustness and stability of quantum classifiers on small biomedical datasets, the system contributes to ongoing research investigating the practical applicability of quantum computing in medical data analysis and early disease detection.

2. Architectural Design

The architectural design of the Hybrid Classical–Quantum Parkinson’s Disease Detection System defines the overall structure of the system, including its major layers, modules, and their interactions. The system is designed as a **modular layered architecture** that integrates classical machine learning components with quantum computing modules within a unified experimental pipeline.

Since the system is implemented as a **Python-based research prototype**, it does not follow traditional web-based architectures such as client–server or microservices models. Instead, it follows a **layered experimental architecture**, where each layer performs a specific role in the machine learning and quantum experimentation workflow. This architectural approach ensures clear separation of responsibilities, allowing the system to remain modular, extensible, and suitable for scientific experimentation.

The architecture integrates **classical data processing, hybrid quantum–classical model training, quantum noise simulation, performance evaluation, and visualization components** into a cohesive framework. The system is designed to allow researchers to configure experiments, process biomedical datasets, train hybrid models, simulate realistic quantum noise environments, and analyze model robustness through statistical evaluation.

The architectural design focuses on the following key objectives:

- **Modularity** – separating system functionality into independent components
- **Extensibility** – enabling the addition of new models, datasets, or quantum circuits
- **Reproducibility** – supporting consistent experimental execution
- **Experimental scalability** – enabling multiple training runs and noise experiments
- **Hybrid integration** – allowing seamless interaction between classical and quantum modules

To achieve these goals, the system architecture is divided into several functional layers that interact sequentially to execute the experimental workflow.

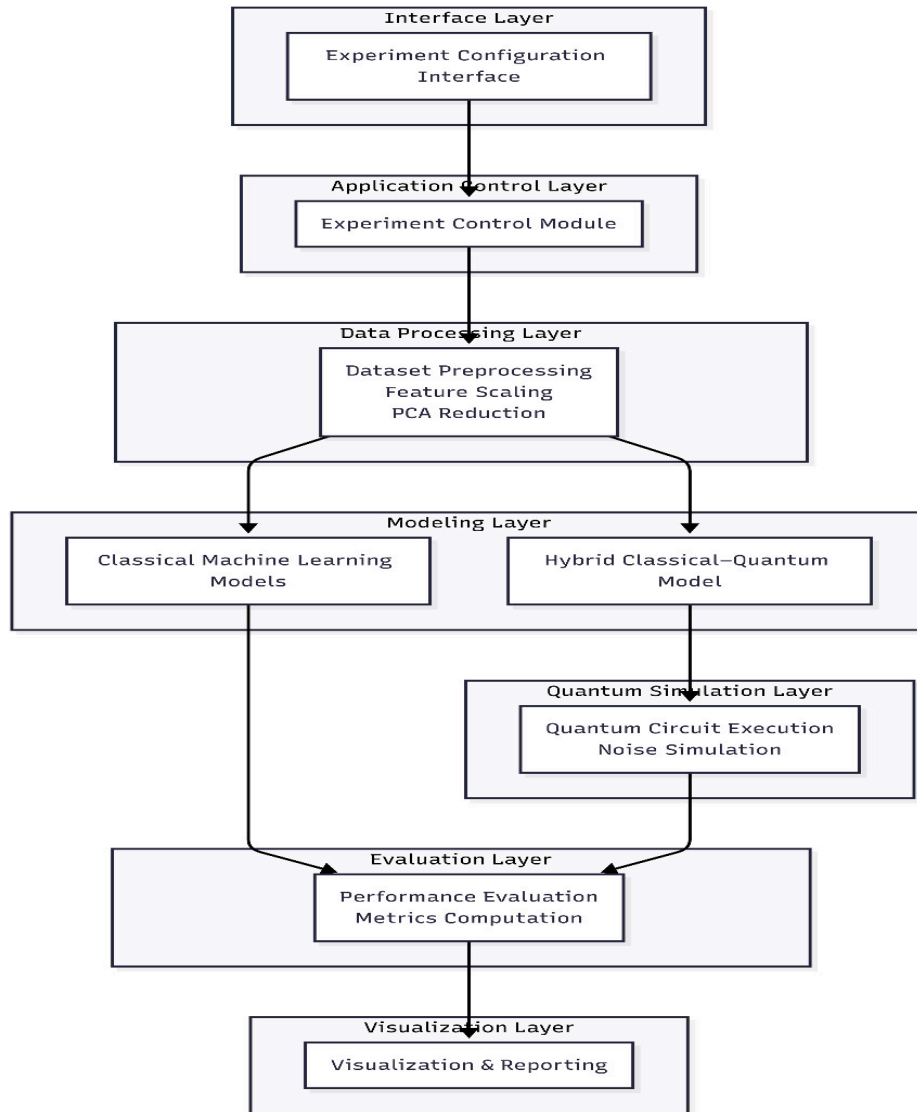


Figure 2.1 : Basic architecture diagram

Fig. 2.1 represents the basic architecture of the proposed system, showing the data processing pipeline from dataset input and preprocessing to model training, noise simulation, performance evaluation, and result visualization.

2.1 Layered Architecture Overview

The system architecture consists of the following layers:

1. User Interaction Layer
2. Application Control Layer
3. Data Processing Layer
4. Hybrid Modeling Layer (Classical + Quantum)
5. Quantum Simulation and Noise Modeling Layer
6. Evaluation and Metrics Layer
7. Visualization and Reporting Layer

Each layer communicates with adjacent layers through structured data pipelines, ensuring that system components remain loosely coupled while maintaining a consistent data flow.

2.2 User Interaction Layer

The User Interaction Layer provides the interface through which the researcher interacts with the system. It allows users to configure experiments, load datasets, select preprocessing methods, and execute model training.

Typical user operations include:

- Loading the Parkinson's dataset
- Configuring preprocessing parameters such as normalization and PCA
- Selecting dataset subset sizes
- Training classical and hybrid quantum models
- Enabling or disabling quantum noise simulation
- Running experimental evaluations
- Viewing graphical visualizations of model performance

This layer simplifies experiment execution by providing a structured workflow for running machine learning and quantum simulations.

2.3 Application Control Layer

The Application Control Layer acts as the central coordinator of the system. It manages the execution of the experimental pipeline and ensures that each stage of the workflow is performed in the correct order.

Responsibilities include:

- Managing experiment configurations
- Controlling the data processing workflow
- Invoking classical and hybrid model training modules
- Executing quantum simulation experiments
- Running repeated experiments for statistical validation
- Coordinating evaluation and visualization components

By centralizing control logic, this layer ensures reproducible and consistent experimental execution.

2.4 Data Processing Layer

The Data Processing Layer prepares the Parkinson's dataset for both classical and quantum machine learning models.

The dataset typically contains a large number of biomedical or acoustic features extracted from voice recordings. Since quantum circuits can only handle a limited number of qubits, dimensionality reduction techniques are required.

Key preprocessing operations include:

- Loading dataset from CSV files
- Data validation and cleaning
- Feature normalization and scaling
- Dimensionality reduction using Principal Component Analysis (PCA)
- Generation of small dataset subsets for experimentation

The output of this layer is a structured and reduced feature dataset compatible with both classical algorithms and quantum circuit encoding.

2.5 Hybrid Modeling Layer (Classical–Quantum)

The Hybrid Modeling Layer forms the computational core of the system. It integrates classical machine learning methods with quantum circuit–based classification.

The hybrid learning pipeline follows a classical–quantum workflow where classical preprocessing prepares features that are then encoded into quantum states for classification.

The modeling layer contains two main components:

Classical Baseline Models

Classical models are implemented to provide performance benchmarks. These models include:

- Support Vector Machine (SVM)
- Random Forest Classifier

These algorithms are implemented using the Scikit-learn library and serve as reference models.

Hybrid Quantum Model

The hybrid model uses a **Variational Quantum Classifier (VQC)** as the primary classification mechanism. The hybrid workflow consists of the following steps:

1. Encoding classical feature vectors into quantum states using rotation gates
2. Passing the encoded data through a parameterized quantum circuit (ansatz)
3. Optimizing circuit parameters using classical optimization algorithms
4. Measuring qubits to produce classification predictions

The hybrid approach combines the representational power of quantum circuits with classical optimization techniques, forming a quantum–classical training loop.

2.6 Quantum Simulation and Noise Modeling Layer

Since real quantum computers are limited and not always accessible, the system executes quantum circuits using high-performance simulators.

This layer supports two experimental environments:

- **Ideal Simulation** – quantum circuits are executed without noise
- **Noisy Simulation** – artificial quantum noise models are injected during circuit execution

The noise simulation module supports several realistic quantum noise models including:

- Bit-flip noise
- Phase-flip noise
- Depolarizing noise
- Amplitude damping

These noise models emulate the behavior of real **Noisy Intermediate-Scale Quantum (NISQ)** hardware, allowing the system to evaluate how quantum noise affects model performance.

2.7 Evaluation and Metrics Layer

The Evaluation Layer measures the predictive performance and robustness of the models.

Classification metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

To evaluate noise robustness, the system also computes statistical and stability metrics such as:

- Mean accuracy across multiple runs
- Standard deviation and variance
- Performance degradation under noise

These metrics help assess the reliability and stability of hybrid quantum models.

2.8 Visualization and Reporting Layer

The Visualization Layer generates graphical outputs and experimental reports that summarize model performance.

Typical visualizations include:

- Classical vs Hybrid model accuracy comparisons
- Ideal vs Noisy quantum performance plots
- Dataset size vs model performance graphs
- Statistical stability analysis charts

Visualization tools used include:

- Matplotlib
- Seaborn

These graphical outputs help researchers interpret experimental results and incorporate findings into research reports.

2.9 Module Interaction Workflow

The interaction between system modules follows a structured experimental pipeline:

1. The user configures an experiment through the User Interaction Layer.
2. The Application Control Layer manages the experimental workflow.
3. The dataset is processed in the Data Processing Layer.
4. Processed data is passed to the Hybrid Modeling Layer for classical and quantum model training.
5. Quantum circuits are executed in the Quantum Simulation Layer with optional noise models.
6. Model predictions are evaluated using classification and statistical metrics.
7. Results are visualized and reported through the Visualization Layer.

This layered architecture ensures that the system remains modular, maintainable, and extensible while enabling controlled experimentation with hybrid quantum machine learning models.

3. User Interface (UI) Design and Component Design

The **User Interface (UI) Design** defines how the researcher interacts with the experimental system to configure experiments, execute model training, and analyze results. Since the system is implemented as a **research-oriented Python desktop application**, the interface focuses on providing a **structured and intuitive workflow for conducting machine learning experiments** rather than supporting large-scale public usage.

The UI is designed with the following goals:

- **Usability** – allowing researchers to easily configure experiments
- **Clarity** – presenting experiment parameters and results in an understandable format
- **Modularity** – separating UI components for easier system maintenance
- **Reproducibility** – enabling consistent experimental setups

The interface guides the user through the complete experimental pipeline including **dataset loading, preprocessing, model training, noise simulation, and result visualization**.

3.1 User Interaction Workflow

The system follows a **step-by-step experimental workflow** where the researcher sequentially configures and executes each stage of the experiment.

Typical interaction steps include:

1. Load Parkinson's disease dataset
2. Configure preprocessing parameters
3. Select dataset subset size (20, 50, or 100 samples)

4. Choose classical or quantum models for training
5. Enable or disable noise simulation
6. Execute model training and evaluation
7. Visualize experimental results and metrics

This structured workflow ensures that experiments are executed in a **controlled and reproducible manner**.

3.2 User Interface Layout

The interface consists of several functional sections that correspond to major experimental modules. Each section provides controls for configuring parameters and viewing outputs.

The main UI sections include:

- Dataset Management Panel
- Experiment Configuration Panel
- Model Training Panel
- Noise Simulation Panel
- Results and Visualization Panel

These sections provide a clear logical organization of experimental tasks.

3.3 UI Architecture

The UI architecture follows a **front-end controller model**, where the interface interacts with backend experimental modules through a central control component. This architecture separates **user interaction logic from computational modules**, improving maintainability and modularity.

3.4 Wireframe Design

The wireframe represents the conceptual layout of the interface components.

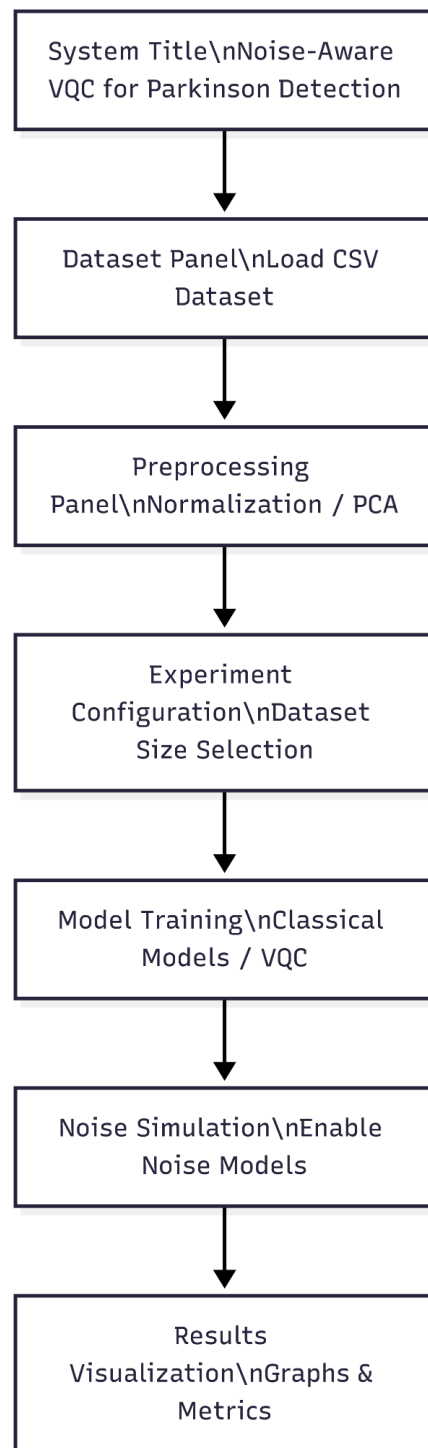


Figure 3.2 : Project wireframe

Fig. 3.2 represents the project wireframe, illustrating the layout of the system interface and the sequential flow of experimental stages, ensuring that users configure and execute the experiment in a structured and error-free manner.

3.5 Front-End Components

The front-end interface is divided into reusable UI components that manage user interactions and display information.

Dataset Loader Component

Purpose

Allows the user to load Parkinson's datasets into the system.

Inputs

- Dataset file (CSV format)

Outputs

- Structured dataset loaded into the system

Functionality

- File selection interface
- Dataset validation
- Dataset preview display

Preprocessing Configuration Component

Purpose

Allows users to configure preprocessing parameters.

Inputs

- Raw dataset

Outputs

- Normalized dataset
- Optional dimensionality-reduced dataset

Functionality

- Feature scaling options
- PCA enable/disable selection
- Dataset subset generation

Experiment Configuration Component

Purpose

Allows the user to configure experimental parameters.

Inputs

- Dataset subset size
- Model selection

Outputs

- Experiment configuration parameters

Functionality

- Select dataset size (20, 50, 100 samples)
- Choose classical or quantum models
- Configure number of experiment runs

Model Training Component

Purpose

Executes training procedures for classical and quantum models.

Inputs

- Processed dataset
- Model parameters

Outputs

- Trained model
- Prediction results

Functionality

- Train classical models (SVM, Random Forest)
- Train Variational Quantum Classifier

Noise Simulation Component

Purpose

Simulates realistic quantum hardware noise during circuit execution.

Inputs

- Quantum circuit parameters
- Noise configuration

Outputs

- Noisy quantum predictions

Functionality

- Enable/disable noise simulation
- Configure noise parameters

Results Visualization Component

Purpose

Displays experimental results and evaluation metrics.

Inputs

- Evaluation results
- Statistical metrics

Outputs

- Graphs and performance tables

Functionality

- Plot accuracy comparisons
- Visualize noise effects
- Display statistical summaries

3.6 Backend Integration

The UI communicates with backend computational modules through an **Application Controller**, which acts as an intermediary between user actions and system functionality.

The backend integration layer connects the UI with:

- Data processing modules
- Classical machine learning models
- Quantum circuit simulation frameworks
- Evaluation and visualization modules

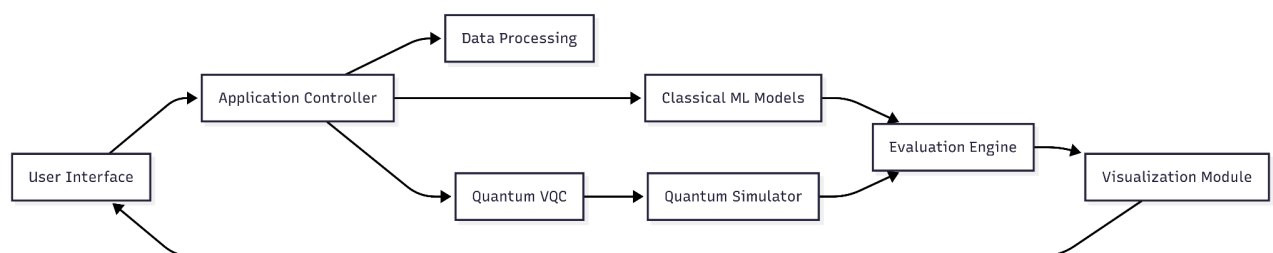


Figure 3.3 : Backend integration flow diagram

Fig. 3.3 represents the backend integration flow diagram showing the interaction between the user interface and backend computational modules, ensuring clear separation between UI and processing logic.

3.7 User Feedback Mechanisms

The system provides feedback to guide the user during experiment execution.

Examples include:

- Dataset successfully loaded notifications
- Training progress indicators
- Error messages for invalid dataset formats
- Experiment completion notifications
- Visualization updates after evaluation

These feedback mechanisms improve usability and prevent configuration errors.

3.8 Accessibility and Usability Considerations

The system interface incorporates several usability considerations to ensure an effective research workflow.

These include:

- Clear labeling of experimental parameters
- Structured step-by-step experiment workflow
- Visual feedback for model training and evaluation
- Organized result visualization panels

The interface prioritizes **clarity and experimental reproducibility**, enabling researchers to conduct controlled machine learning experiments efficiently.

4. Class Diagram and Classes

The **Class Diagram** represents the static structure of the system by defining the main classes, their attributes, methods, and relationships. It provides a detailed view of how different components of the system interact and collaborate to perform tasks such as dataset preprocessing, model training, quantum simulation, and performance evaluation.

For the **Noise-Aware Variational Quantum Classifier for Parkinson's Disease Detection**, the class structure follows a **modular object-oriented design** where each module of the system is represented as a separate class. This design improves maintainability, reusability, and extensibility of the experimental framework.

4.1 Class Diagram

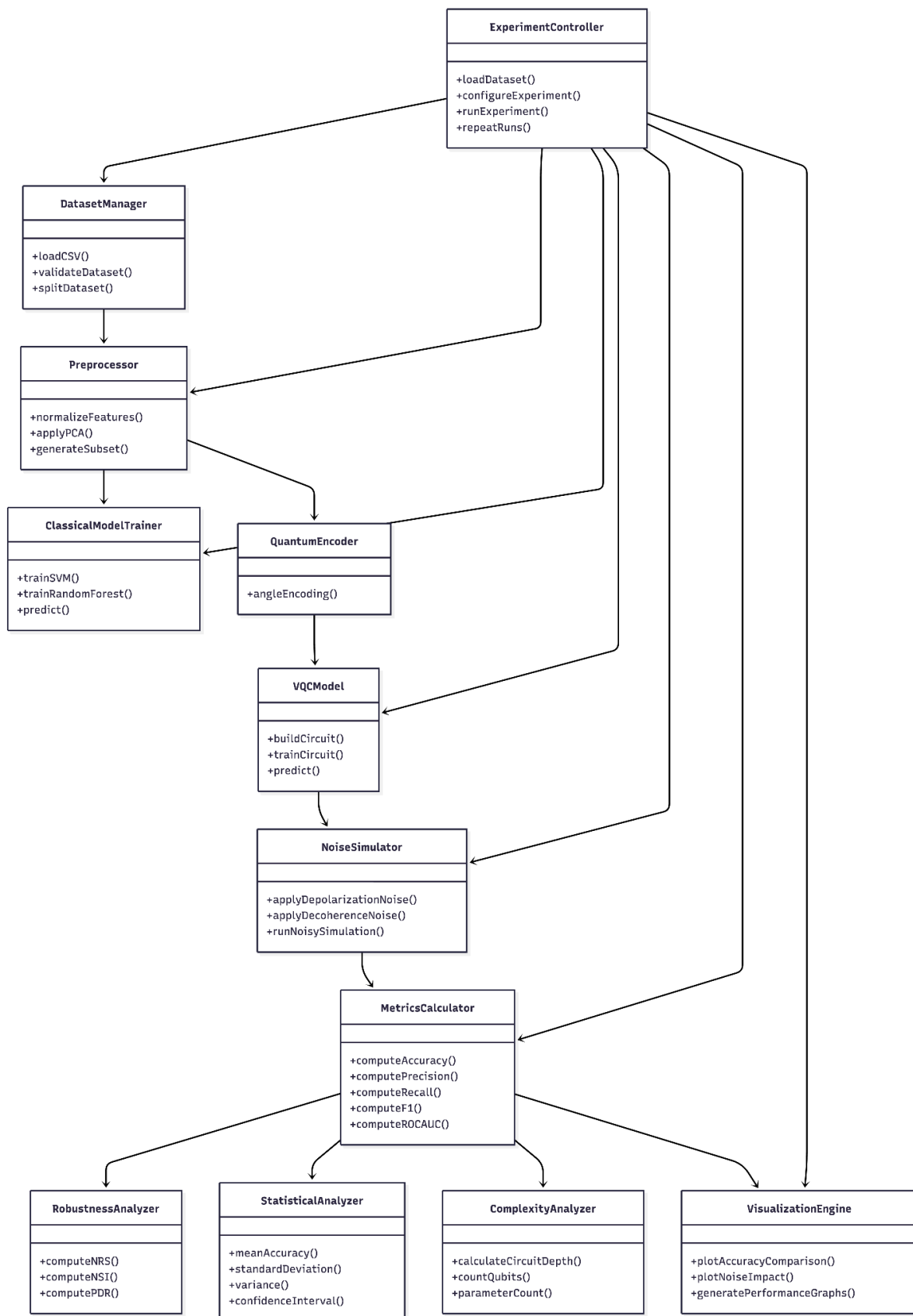


Figure 4.1 : Class Diagram of the Noise-Aware VQC for Parkinson's Disease Detection

The class architecture is organized into the following groups:

1. Core System Control Classes
2. Data Management Classes
3. Modeling Classes
4. Quantum Simulation Classes
5. Evaluation and Metrics Classes
6. Visualization and Reporting Classes

These classes collectively implement the entire machine learning experimentation pipeline.

Fig. 4.1 represents the class diagram of the proposed system, showing the main classes and their relationships involved in dataset management, preprocessing, model training, noise simulation, evaluation, and visualization.

4.2 Core System Control Classes

ExperimentController

The **ExperimentController** class acts as the central coordinator for the entire experimental pipeline. It manages experiment configuration, dataset preparation, model training, and evaluation execution.

Attributes

- experimentConfig
- datasetSize
- numRuns

Methods

- loadDataset() – loads the dataset into the system
- configureExperiment() – sets experiment parameters such as dataset size and noise settings
- runExperiment() – executes the full training and evaluation pipeline
- repeatRuns() – performs multiple experimental runs for statistical analysis

This class orchestrates interactions between all major system modules.

4.3 Data Management Classes

DatasetManager

The **DatasetManager** class handles dataset loading and validation.

Attributes

- datasetPath
- rawDataset

Methods

- loadCSV() – loads dataset from CSV file
- validateDataset() – checks dataset integrity
- splitDataset() – divides dataset into training and testing sets

Preprocessor

The **Preprocessor** class performs data transformation operations required before model training.

Attributes

- normalizedData
- reducedFeatures

Methods

- normalizeFeatures() – scales feature values
- applyPCA() – performs dimensionality reduction
- generateSubset() – creates dataset subsets (20, 50, 100 samples)

4.4 Modeling Classes

ClassicalModelTrainer

This class implements baseline classical machine learning algorithms.

Attributes

- svmModel
- randomForestModel

Methods

- trainSVM() – trains Support Vector Machine classifier
- trainRandomForest() – trains Random Forest classifier
- predict() – generates predictions for test samples

QuantumEncoder

The **QuantumEncoder** class converts classical feature vectors into quantum states.

Methods

- `angleEncoding()` – encodes classical data using rotation gates

This class prepares classical data for quantum circuit processing.

VQCModel

The **VQCModel** class implements the Variational Quantum Classifier.

Attributes

- `numQubits`
- `circuitDepth`
- `parameters`

Methods

- `buildCircuit()` – constructs parameterized quantum circuit
- `trainCircuit()` – optimizes circuit parameters using classical optimizers
- `predict()` – generates classification results based on quantum measurement outcomes

This class represents the core quantum machine learning component.

4.5 Quantum Simulation Classes

NoiseSimulator

The **NoiseSimulator** class simulates realistic quantum noise conditions.

Attributes

- `noiseModel`
- `noiseLevel`

Methods

- `applyDepolarizationNoise()` – simulates depolarization errors
- `applyDecoherenceNoise()` – simulates qubit decoherence
- `runNoisySimulation()` – executes quantum circuits under noisy conditions

This class enables the system to evaluate model performance under **NISQ-like hardware constraints**.

4.6 Evaluation and Metrics Classes

MetricsCalculator

This class computes core classification performance metrics.

Methods

- computeAccuracy()
- computePrecision()
- computeRecall()
- computeF1()
- computeROCAUC()

These metrics evaluate classification performance of both classical and quantum models.

RobustnessAnalyzer

The **RobustnessAnalyzer** evaluates how well the quantum model handles noise.

Methods

- computeNRS() – Noise Robustness Score
- computeNSI() – Noise Sensitivity Index
- computePDR() – Performance Degradation Rate

These metrics quantify the impact of quantum noise on model performance.

StatisticalAnalyzer

The **StatisticalAnalyzer** evaluates experimental stability across multiple runs.

Methods

- meanAccuracy()
- standardDeviation()
- variance()
- confidenceInterval()

These metrics ensure reliability and reproducibility of experimental results.

ComplexityAnalyzer

The **ComplexityAnalyzer** evaluates the structural complexity of the quantum model.

Methods

- calculateCircuitDepth()
- countQubits()
- parameterCount()

4.7 Visualization and Reporting Classes

VisualizationEngine

The **VisualizationEngine** generates graphical outputs that help analyze experimental results.

Methods

- `plotAccuracyComparison()` – compares classical and quantum model performance
- `plotNoiseImpact()` – visualizes effect of noise on VQC performance
- `generatePerformanceGraphs()` – produces graphs for dataset size vs accuracy

Visualization is implemented using **Matplotlib and Seaborn** libraries.

The class diagram organizes the system into well-defined components responsible for dataset management, model training, quantum simulation, performance evaluation, and visualization.

This modular object-oriented design ensures that the system remains:

- **Extensible** for adding new quantum circuits or datasets
- **Maintainable** through separation of responsibilities
- **Reusable** for future quantum machine learning experiments

The defined class structure supports the complete workflow required to evaluate the feasibility, robustness, and stability of a **Noise-Aware Variational Quantum Classifier for Parkinson's disease detection** under realistic noisy quantum simulation environments.

5. Sequence Diagram

The Sequence Diagram illustrates the dynamic behavior of the system by showing how different components interact over time to complete an experiment. It represents the sequence of operations executed during the processing pipeline, including dataset loading, preprocessing, model training, quantum simulation, evaluation, and result visualization.

Unlike traditional software systems where the primary actor may be a user or external entity, the sequence diagram for this project focuses on the **System Pipeline** as the initiating entity. This approach better reflects the automated experimental workflow of the system, where the pipeline coordinates interactions between modules responsible for machine learning, quantum circuit simulation, and evaluation.

The sequence diagram helps in understanding:

- The order of interactions between modules
- The data flow during experiment execution
- The integration of classical and quantum modeling components
- The evaluation process under ideal and noisy quantum simulation conditions

5.1 Sequence Diagram Overview

The system pipeline performs the following sequential operations:

1. Initialize experiment configuration
2. Load Parkinson's dataset
3. Perform dataset preprocessing
4. Train classical baseline models
5. Encode data into quantum states
6. Build and train the Variational Quantum Classifier
7. Execute quantum simulation under ideal and noisy conditions
8. Evaluate model performance using multiple evaluation metrics
9. Perform statistical and robustness analysis
10. Generate visualizations and experimental reports

These interactions occur between different system modules designed in the architectural and class design sections.

5.2 Sequence Diagram Components

The sequence diagram includes the following participants:

Component	Description
System Pipeline	Initiates and controls the experimental workflow
Dataset Manager	Loads dataset from CSV files
Preprocessing Module	Performs feature normalization and dataset preparation
Classical Model Trainer	Trains classical ML models (SVM, Random Forest)
Quantum Encoder	Converts classical data into quantum states
VQC Model	Builds and trains the variational quantum circuit
Noise Simulator	Executes circuits with realistic quantum noise
Metrics Engine	Computes classification performance metrics
Statistical Analyzer	Performs stability and variance analysis
Visualization Engine	Generates graphs and experimental reports

Table 5.1: Description of the components in the sequence diagram

Table 5.1 presents the description of the components involved in the sequence diagram of the proposed system. It outlines the roles of each module in the experimental pipeline, including dataset management, preprocessing, classical and quantum model training, noise simulation, performance evaluation, statistical analysis, and result visualization.

5.3 Sequence Diagram

The following diagram represents the complete sequence diagram for the system.

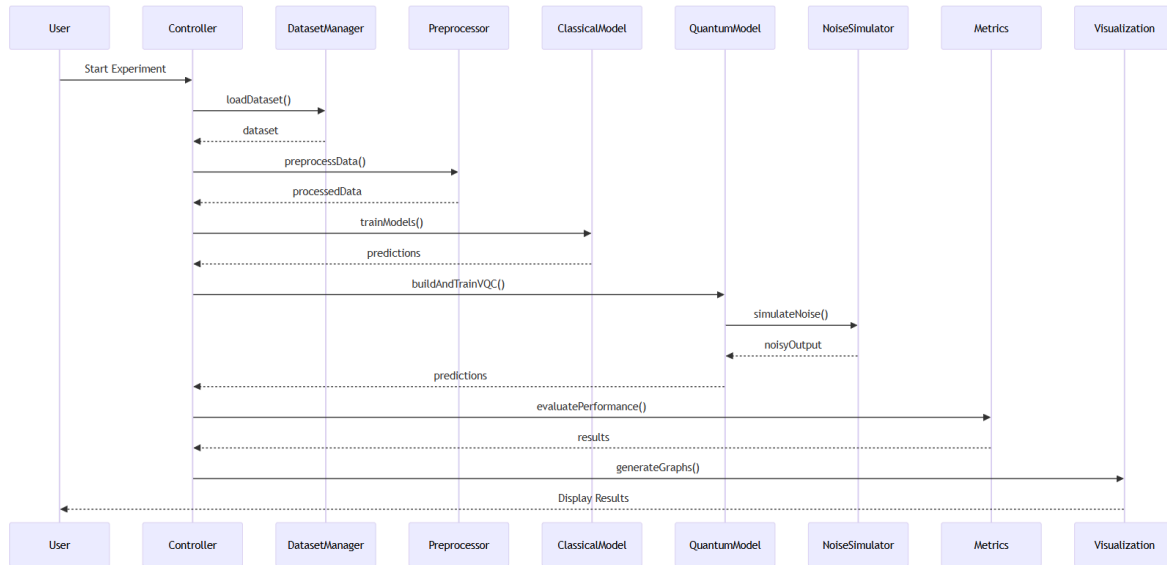


Figure 5.1 : Sequence Diagram of the Noise-Aware VQC for Parkinson's Disease Detection

Fig. 5.1 represents the sequence diagram of the proposed system, showing the interaction and flow of operations between different modules during the execution of the Parkinson's disease detection experiment.

5.4 Interaction Flow Explanation

The execution of the sequence diagram follows a structured experimental pipeline.

• Dataset Initialization

The System Pipeline begins by requesting the Dataset Manager to load the Parkinson's dataset from a CSV file. Once the dataset is successfully loaded, it is passed to the preprocessing module.

• Data Preprocessing

The Preprocessing Module performs feature normalization and optional dimensionality reduction. It also generates smaller dataset subsets for experimental evaluation.

• Classical Model Training

The Classical Model Trainer receives the processed dataset and trains baseline models such as Support Vector Machines and Random Forest classifiers. The predictions generated by these models are returned to the system pipeline.

- **Quantum Feature Encoding**

Next, the Quantum Encoder converts classical feature vectors into quantum states using angle encoding techniques.

- **Variational Quantum Classifier Construction**

The encoded quantum data is passed to the VQC Model, where a parameterized quantum circuit is constructed. The circuit parameters are optimized using classical optimization algorithms.

- **Quantum Simulation**

The trained quantum circuit is executed by the Noise Simulator, which performs two types of simulations:

- Ideal (noise-free) simulation
- Noisy simulation using NISQ noise models

This step evaluates how quantum hardware noise affects the classifier's performance.

- **Performance Evaluation**

The results from classical and quantum models are passed to the Metrics Engine, which computes classification metrics such as accuracy, precision, recall, F1-score, and ROC-AUC.

- **Statistical Analysis**

The Statistical Analyzer then calculates stability metrics such as mean accuracy, variance, and standard deviation across multiple experimental runs.

- **Visualization and Reporting**

Finally, the Visualization Engine generates graphical outputs and experimental reports that summarize the performance of classical and quantum models under different experimental conditions.

The sequence diagram demonstrates how the System Pipeline coordinates interactions between all major system components to execute the machine learning and quantum simulation workflow. By organizing operations into a structured sequence, the system ensures consistent experimental execution and reliable evaluation of the Noise-Aware Variational Quantum Classifier under realistic noisy simulation environments.

This diagram provides a clear representation of the runtime behavior of the system and highlights the integration of classical machine learning models, quantum circuits, noise simulation, and statistical evaluation within a unified experimental framework.

6. Security & Performance Considerations

Security and performance are important aspects of the system design to ensure that the experimental framework operates reliably, protects sensitive biomedical data, and maintains efficient computational performance during machine learning and quantum simulation experiments.

Although the proposed system is a **research-oriented prototype** and not intended for real-world clinical deployment, it still incorporates appropriate security mechanisms and performance optimization strategies to ensure safe handling of data and efficient execution of experiments.

The system focuses on two main areas:

1. **Security Measures** – protecting datasets, experimental configurations, and system access
2. **Performance Optimization** – ensuring efficient model training, quantum simulation, and evaluation

6.1 Security Considerations

The Parkinson's dataset used in the system contains biomedical information, which requires responsible handling even in a research environment. The system incorporates several security measures to protect data integrity and system access.

6.1.1 Data Protection

The system stores datasets locally in **CSV format**, and only authorized system users can access or modify the dataset files.

Security measures include:

- Restricted access to dataset files within the project directory
- Read-only dataset loading during experiment execution
- Validation of dataset inputs before processing

These mechanisms prevent accidental modification or corruption of experimental datasets.

6.1.2 Input Validation

To maintain system integrity, the system performs validation checks on all dataset inputs and configuration parameters before executing experiments.

Validation procedures include:

- File format verification (CSV format validation)
- Dataset structure validation (checking required feature columns)
- Detection of missing or corrupted data values

Input validation ensures that invalid data does not disrupt the experimental workflow.

6.1.3 Access Control

Since the system is intended for research use, access to the system is restricted to authorized users such as researchers or project developers.

Basic access control measures include:

- Local system authentication through operating system login credentials
- Restricted modification access to source code and dataset files
- Controlled execution of experimental scripts

These controls ensure that only authorized users can run experiments or modify system configurations.

6.1.4 Data Privacy

Although the dataset used is publicly available (such as the Parkinson's dataset from research repositories), the system maintains responsible data handling practices.

Privacy considerations include:

- Avoiding storage of personally identifiable information (PII)
- Using anonymized biomedical feature datasets
- Limiting dataset sharing outside the experimental environment

These practices align with ethical standards for biomedical data handling.

6.1.5 System Integrity

To ensure reliable operation of the experimental framework, the system implements safeguards that maintain system stability.

Examples include:

- Error handling mechanisms during model training and simulation
- Logging of experiment execution for debugging and traceability
- Protection against invalid experimental configurations

These safeguards prevent runtime failures and improve system reliability.

6.2 Performance Considerations

Quantum circuit simulation and repeated experimental runs can require significant computational resources. Therefore, the system incorporates performance optimization techniques to ensure efficient experiment execution.

6.2.1 Efficient Data Processing

Data preprocessing operations are optimized using efficient numerical computing libraries such as:

- **NumPy** for numerical operations
- **Pandas** for structured data processing

These libraries allow fast manipulation of dataset features and efficient preprocessing operations.

6.2.2 Optimized Machine Learning Training

The classical machine learning models are implemented using **Scikit-learn**, which provides optimized algorithms for training models efficiently on small to medium-sized datasets.

Performance improvements include:

- Efficient implementation of Support Vector Machines and Random Forest algorithms
- Use of vectorized operations for faster model training
- Controlled dataset sizes (20, 50, 100 samples) to ensure manageable computational requirements

6.2.3 Quantum Circuit Optimization

Quantum circuit simulation is computationally expensive, particularly when simulating noise models. To reduce simulation overhead, the system applies the following optimization strategies:

- Limiting the number of qubits used in the quantum circuit
- Controlling circuit depth to reduce simulation complexity
- Using lightweight variational ansatz structures
- Selecting efficient classical optimizers such as COBYLA

These strategies ensure that quantum experiments remain computationally feasible on classical hardware.

6.2.4 Parallel Experiment Execution

The system supports executing **multiple experimental runs** for statistical analysis. To improve efficiency, experiments can be executed using parallel processing techniques when supported by the computing environment.

Benefits include:

- Faster evaluation across multiple runs
- Reduced total experiment runtime
- Improved scalability for additional experiments

6.2.5 Memory Management

Efficient memory usage is important when working with multiple experimental runs and quantum circuit simulations.

Memory optimization techniques include:

- Efficient storage of intermediate results
- Clearing unused variables during execution
- Avoiding redundant dataset copies in memory

These strategies help maintain stable system performance during repeated experiments.

6.2.6 Visualization Efficiency

Visualization operations are optimized to ensure that graphs and performance plots are generated efficiently without excessive memory consumption.

The system uses optimized visualization libraries such as:

- **Matplotlib**
- **Seaborn**

These libraries allow efficient rendering of graphs for performance analysis.

6.3 Scalability Considerations

Although the system primarily focuses on **small dataset experiments**, the architecture allows future scalability.

Possible scalability improvements include:

- Integration with more advanced quantum simulators
- Addition of larger biomedical datasets
- Extension to additional quantum machine learning models

The modular design ensures that these extensions can be implemented without major changes to the core architecture.

The system incorporates essential **security and performance mechanisms** to ensure safe and efficient operation of the experimental framework. Security measures such as input validation, controlled dataset access, and system integrity checks help protect biomedical data and maintain system stability.

At the same time, performance optimization techniques—including efficient data processing, optimized classical and quantum model training, and controlled experiment execution—ensure that the system can perform multiple experiments efficiently within the computational limits of classical hardware.

These considerations ensure that the **Noise-Aware Variational Quantum Classifier experimental system** remains reliable, efficient, and suitable for conducting controlled quantum machine learning research experiments.

7. Technology Stack Selection

The **Technology Stack Selection** defines the programming languages, frameworks, libraries, simulation environments, and supporting tools used to implement the **Noise-Aware Variational Quantum Classifier for Parkinson's Disease Detection**.

Since the system is designed as a **Python-based research prototype**, the technology stack focuses on scientific computing, machine learning frameworks, and quantum simulation tools rather than traditional web development stacks such as MERN. The selected technologies enable efficient data processing, classical machine learning modeling, quantum circuit simulation, statistical analysis, and visualization.

The technology stack is selected based on the following criteria:

- Compatibility with **quantum machine learning frameworks**
- Support for **scientific computing and data analysis**
- Availability of **quantum simulation environments**
- Ease of integration between classical and quantum components
- Suitability for **research experimentation**

According to the system requirements and project synopsis, the system operates on a **Python environment using quantum SDKs and classical machine learning libraries** running on classical computing hardware.

7.1 Programming Language

Python

Python is selected as the primary programming language for the system implementation.

Reasons for Selection

- Extensive ecosystem for **machine learning and scientific computing**
- Native support for **quantum computing frameworks**
- Easy integration between classical and quantum libraries
- Large developer and research community

Role in the System

Python is used to implement:

- Data preprocessing modules
- Classical machine learning models
- Quantum circuit construction

- Noise simulation
- Performance evaluation modules
- Visualization and reporting tools

The project requires **Python 3.9 or higher** to ensure compatibility with quantum machine learning frameworks and scientific libraries.

7.2 Quantum Machine Learning Frameworks

Qiskit

Qiskit is an open-source quantum computing framework used to design, simulate, and execute quantum circuits.

Key Features

- Quantum circuit construction tools
- Parameterized quantum circuits (variational circuits)
- Noise modeling support
- Integration with quantum simulators

Role in the System

Qiskit is used to:

- Build **Variational Quantum Circuits (VQC)**
- Implement quantum feature encoding
- Integrate noise models
- Execute quantum simulations

PennyLane

PennyLane is a quantum machine learning framework that enables integration between quantum circuits and classical machine learning workflows.

Key Features

- Hybrid quantum-classical model development
- Integration with machine learning frameworks
- Automatic differentiation for quantum circuits

Role in the System

PennyLane supports:

- Implementation of **hybrid classical–quantum models**
- Training variational quantum circuits
- Efficient gradient-based optimization

These frameworks allow the system to perform realistic simulations of quantum machine learning algorithms.

7.3 Classical Machine Learning Libraries

Scikit-learn

Scikit-learn is used to implement classical machine learning baseline models,

Supported Models

- Support Vector Machine (SVM)
- Random Forest Classifier

Role in the System

Scikit-learn provides:

- Efficient model training
- Built-in evaluation metrics
- Dataset splitting and validation tools

These models act as **baseline benchmarks** for evaluating the performance of the Variational Quantum Classifier.

7.4 Data Processing Libraries

NumPy

NumPy is used for high-performance numerical computations.

Role in the System

- Matrix operations
- Vectorized numerical computations
- Efficient data manipulation

Pandas

Pandas is used for structured dataset processing.

Role in the System

- Loading datasets from CSV files
- Data cleaning and preprocessing
- Feature transformation

These libraries allow efficient handling of biomedical dataset features.

SciPy

SciPy provides scientific computing tools used for advanced numerical operations.

Role in the System

- Statistical analysis
- Optimization algorithms
- Scientific computation functions

7.5 Quantum Simulation Environment

Qiskit Aer Simulator

The **Qiskit Aer Simulator** is used to execute quantum circuits in a simulated environment.

Features

- Ideal (noise-free) simulation
- Noise model integration
- High-performance simulation backend

Role in the System

- Execute variational quantum circuits
- Simulate noisy quantum environments
- Evaluate model behavior under NISQ conditions

PennyLane lightning.qubit

The **lightning.qubit** backend is a high-performance simulator for quantum circuits.

Role in the System

- Faster quantum circuit execution
- Efficient simulation of variational circuits
- Integration with PennyLane workflows

These simulators allow quantum experiments to run on classical hardware without requiring real quantum computers.

7.6 Data Visualization Tools

Matplotlib

Matplotlib is used to generate graphs and plots representing experimental results.

Role in the System

- Noise impact analysis
- Dataset size vs performance plots

Seaborn

Seaborn is used to enhance visualization quality and statistical graph representation.

Role in the System

- Statistical distribution plots
- Comparative model performance charts

Visualization tools help present experimental findings clearly for academic analysis.

7.7 Development Environment

The system is developed using a standard Python development environment.

- Jupyter Notebook (for experimentation)
- VS Code or PyCharm (for development)
- Git for version control

These tools support efficient coding, debugging, and experimental documentation.

7.8 Hardware Requirements

The system is designed to operate on **standard laptop hardware**, since quantum circuits are simulated rather than executed on real quantum hardware.

Minimum hardware requirements include:

- Intel i5 or equivalent processor
- Minimum **8 GB RAM**
- At least **20 GB free storage**

Quantum circuits are executed through simulation rather than physical quantum processors.

7.9 Operating System and Execution Environment

The system supports the following operating systems:

- Windows
- Linux

The application runs locally and does not require external network communication unless optional cloud quantum services are used.

7.10 Technology Stack Summary

Category	Technology
Programming Language	Python 3.9+
Quantum ML Frameworks	Qiskit, PennyLane
Classical ML	Scikit-learn
Data Processing	NumPy, Pandas, SciPy
Quantum Simulation	Qiskit Aer, PennyLane, lightning.qubit
Visualization	Matplotlib, Seaborn
Development Tools	Jupyter Notebook, VS Code
Operating System	Windows / Linux

Table 7.1: Technology Stack

Table 7.1 represents the selected technology stack, that enables seamless integration of **classical machine learning and quantum machine learning components** within a unified experimental framework. Python serves as the core programming environment, while specialized libraries such as Qiskit and PennyLane support the development of quantum circuits and noise-aware simulations.

Supporting libraries such as NumPy, Pandas, and Scikit-learn provide efficient tools for data processing and classical model benchmarking, while visualization tools enable clear results.

Together, this technology stack provides a robust environment for implementing and evaluating a **Noise-Aware Variational Quantum Classifier for Parkinson's Disease Detection under realistic noisy quantum simulation conditions**.

8. Scalability & Reliability Planning

Scalability and reliability planning ensure that the system can **handle increasing computational workloads, maintain consistent performance, and produce stable experimental results** even as the system evolves.

Although the **Noise-Aware Variational Quantum Classifier (VQC) system** is designed as a **research-oriented desktop experimental application**, the architecture is designed in a way that allows future extension to larger datasets, more complex quantum circuits, and distributed computing environments.

The planning for scalability and reliability focuses on the following aspects:

1. Computational scalability
2. Dataset scalability
3. Quantum simulation scalability
4. Reliability and reproducibility of experiments
5. Resource management and optimization
6. Potential distributed or cloud-based extensions

These considerations ensure that the system remains robust, efficient, and suitable for continued research experimentation.

8.1 Computational Scalability

The computational complexity of the system primarily arises from two operations:

- Classical machine learning model training
- Quantum circuit simulation

Quantum circuit simulation grows exponentially with the number of qubits, which creates scalability limitations when running on classical hardware.

To address this challenge, the system implements the following strategies:

- Limiting the number of qubits to **approximately 6–8 qubits** to control exponential simulation cost
- Controlling circuit depth to prevent excessive computational overhead
- Using efficient quantum simulators such as **Qiskit Aer** and **PennyLane lightning.qubit**

These constraints ensure that the system remains computationally feasible on standard hardware while still allowing meaningful experimentation.

8.2 Dataset Scalability

The system is designed primarily for **small biomedical datasets**, with controlled experimental sizes such as:

- 20 samples
- 50 samples
- 100 samples

However, the architecture allows future extension to larger datasets.

To support dataset scalability, the system incorporates:

- Modular preprocessing pipelines
- Efficient data handling using **NumPy** and **Pandas**
- Flexible dataset loading from CSV files

This design allows the dataset size to be increased with minimal modification to the overall system architecture.

8.3 Experiment Scalability

The experimental framework is designed to support **multiple training runs** for statistical validation and robustness analysis.

To ensure scalability of experiments:

- Experimental runs are executed in independent iterations
- Results from each run are stored and aggregated later
- Statistical analysis modules compute stability metrics across runs

Future improvements may include **parallel execution of experimental runs**, which would significantly reduce total experiment runtime.

8.4 Resource Optimization

Efficient resource management is important for maintaining scalability in a research environment.

The system implements several resource optimization techniques.

Memory Optimization

- Use of efficient numerical arrays via **NumPy**
- Avoiding redundant data duplication in memory
- Clearing unused variables during experiment execution

Computational Optimization

- Vectorized operations for faster numerical processing
- Lightweight variational quantum circuits
- Efficient classical optimization algorithms

These optimizations allow the system to run multiple experiments without excessive memory.

8.5 Reliability and Reproducibility

Reliability is essential for scientific research systems, particularly when evaluating experimental machine learning models.

The system ensures reliability through the following mechanisms:

Reproducible Experiments

Experiments use **fixed random seeds** to ensure that results can be reproduced across different runs.

Error Handling

The system includes error handling mechanisms for potential issues such as:

- Invalid dataset formats
- Model training convergence failures
- Quantum circuit configuration errors

Logging and Experiment Tracking

Experimental results and configurations are recorded for future analysis. This allows researchers to:

- Trace experiment settings
- Compare results across different configurations
- Reproduce experiments when required

These measures ensure that the system produces **consistent and scientifically reliable results**.

8.6 Fault Tolerance

Although the system operates locally, basic fault tolerance measures are included to prevent failures from interrupting experimental workflows.

Examples include:

- Graceful handling of simulation errors
- Validation of circuit parameters before execution
- Recovery mechanisms when experiments fail due to invalid configurations

These safeguards help maintain system stability during experimentation.

8.7 Potential Cloud and Distributed Extensions

While the current implementation runs locally, the system architecture can be extended to support **distributed or cloud-based computing environments**.

Possible future scalability improvements include:

Cloud-Based Quantum Simulation

Integration with cloud quantum platforms such as:

- IBM Quantum services
- Cloud-hosted quantum simulators

This would allow larger experiments and more realistic hardware testing.

Distributed Experiment Execution

Multiple experiments could be executed in parallel using:

- Distributed computing clusters
- Cloud computing resources

This would significantly reduce experimental runtime when performing large-scale evaluations.

8.8 Load Management

Even though the system does not serve external users or web traffic, internal workload management is still important.

The system manages internal computational load through:

- Controlled dataset sizes
- Limiting circuit complexity
- Sequential execution of computational tasks

These mechanisms prevent excessive system resource usage during experiments.

8.9 System Reliability Summary

The system achieves reliability through:

- Controlled experiment configuration
- Robust preprocessing and validation steps
- Error handling and logging mechanisms
- Reproducible experiment design

At the same time, scalability is supported through modular architecture, efficient data processing, and potential extensions to distributed computing environments.

The **Noise-Aware Variational Quantum Classifier experimental system** is designed to maintain stable and efficient operation while supporting future expansion of experiments and computational workloads.

Scalability strategies focus on controlling quantum circuit complexity, optimizing computational resources, and enabling future distributed execution. Reliability planning ensures that experimental results remain reproducible, stable, and scientifically valid.

These design considerations ensure that the system can continue to support **advanced research experimentation in quantum machine learning under realistic noisy simulation conditions**.

9. Conclusion

The **System Design Document** for the *Noise-Aware Variational Quantum Classifier for Parkinson's Disease Detection* provides a comprehensive blueprint for implementing a research-oriented quantum machine learning system. The design focuses on integrating classical machine learning techniques with quantum circuit-based models to evaluate the feasibility and robustness of quantum approaches in small biomedical datasets.

The proposed system architecture adopts a **modular layered design**, enabling clear separation of responsibilities across different modules such as data preprocessing, classical model training, quantum circuit construction, noise simulation, evaluation, and visualization. This modular design improves maintainability, extensibility, and ease of experimentation. The architecture supports a structured experimental pipeline where data flows sequentially through preprocessing, modeling, simulation, and evaluation stages.

The system incorporates both **classical machine learning models and a Variational Quantum Classifier (VQC)** to establish a rigorous comparative framework. Classical baseline models such as Support Vector Machines and Random Forest provide reference performance levels, while the VQC enables exploration of quantum feature representations and parameterized quantum circuits under realistic noisy conditions.

An important contribution of the system design is the integration of **noise-aware quantum simulation**, which enables the study of model behavior under Noisy Intermediate-Scale

Quantum (NISQ) conditions. By comparing model performance under ideal and noisy simulations, the system provides insights into the robustness and stability of quantum machine learning models.

The design also includes detailed specifications for **evaluation metrics, data flow mechanisms, component interactions, and system classes**, ensuring that the experimental pipeline can be executed in a structured and reproducible manner. Statistical analysis modules further enhance the scientific rigor of the experiments by enabling stability analysis across multiple training runs.

From a technological perspective, the selected stack—comprising **Python, Qiskit, PennyLane, Scikit-learn, NumPy, Pandas, and visualization libraries**—provides a powerful environment for implementing hybrid classical–quantum machine learning workflows. The use of quantum simulators allows experiments to be conducted on classical hardware without requiring access to physical quantum devices.

Overall, the system design establishes a robust experimental framework for investigating the applicability of **quantum machine learning in healthcare analytics**, specifically for Parkinson’s disease detection using limited datasets. The proposed architecture supports controlled experimentation, enabling researchers to systematically evaluate the impact of quantum noise, dataset size, and circuit complexity on model performance.

The implementation of this system is expected to provide valuable insights into the **feasibility, robustness, and stability of noise-aware quantum classifiers**, contributing to the growing body of research exploring the practical applications of quantum computing in machine learning and biomedical data analysis.